Laravel in Production

Architecture, Performance, and Operations at Scale

Nuruzzaman Milon

Laravel in Production

Architecture, Performance, and Operations at Scale

Nuruzzaman Milon

This book was written in Markdown and produced with Ibis Next, using mPDF for PDF generation and CommonMark for HTML rendering.

First published: July 2026

To my family, for making every effort worthwhile.

Preface

Who this book is for

This book is written for a mid-to-senior Laravel engineer who already knows how to build a Laravel application and is now being asked to keep one alive - the person a growing team hands the architecture decisions, the performance incident, and the migration that has to happen without a maintenance window. It assumes you already know how routing, Eloquent, queues, and testing work in Laravel; it doesn't stop to explain the framework. Every page is instead about the layer of decisions that sits on top of it: how you structure a codebase as more than one team starts touching it, how you keep it fast once traffic outgrows what fits comfortably on one database, how you keep it correct when the same request can arrive twice, how you keep it available when a dependency you don't control goes down, and how you run all of that at 3 a.m. on a Tuesday without it becoming a story you're still telling a year later.

If you're looking for a Laravel tutorial - how to define a route, how to write a migration, how Eloquent relationships work - this isn't that book, and there are excellent ones already. If you're the engineer who's outgrown those books and is now staring at a monorepo, a queue backlog, or a balance that doesn't add up, keep reading.

How to read it

The book is organized into eight parts, and the parts are ordered by dependency rather than importance, so reading front to back works, but it isn't the only sane way through it. Part 1 (Foundations) and Part 2 (Performance Engineering) build vocabulary that later parts assume - "shared kernel," "N+1," "backfill," and so on. Part 5 (Observability & Monitoring) is worth reading earlier than its page number suggests, because "correlation ID," "SLI/SLO/SLA," and "burn rate" get reused heavily in Parts 4, 6, and 7. Beyond that, Parts 3 through 8 are largely independent of each other, so if you picked this book up because of a live incident with your queues, your database failover, or your CI pipeline, it's reasonable to skip straight to the relevant chapter. Chapter 31, the capstone, assumes you've read - or at least skimmed - everything before it: it's a single worked incident that touches nearly every decision in the book, and it's meant to be read last.

Each chapter follows the same shape. It opens with a concrete production failure - drawn from about thirteen years of real incidents, anonymized - then works through the pattern that would have prevented or fixed it, and closes with a short "Chapter recap" you can scan later without rereading the whole chapter. Not every one of those incidents happened in a Laravel project; over those years I've worked across many languages and frameworks, and where the original system wasn't Laravel, I've translated the failure into a Laravel setting so it can be demonstrated in the stack this book is about.

About the examples

Every incident described in this book actually happened in real production systems. The failure modes, the trade-offs, and the fixes are drawn from lived experience over thirteen years as a professional software engineer. What isn't real is any identifying detail: company names, product names, team names, exact numbers, and anything else that could point back to a specific employer or client has been changed, generalized, or recombined. The named fictional businesses that recur throughout the book - Northfield (an order-processing platform), Vaultly (a wallet service), Reelcast (a livestreaming and short-video app), and others - are stand-ins for this kind of company, not case studies of any single one; they exist because those are the systems that reliably produce the failure modes worth discussing, not because they map one-to-one onto any company you could name. Nothing here discloses a trade secret, a proprietary architecture, or confidential information belonging to any organization I've worked with; the anonymization exists specifically to protect that while keeping the underlying lesson intact. The architecture decisions and opinions in these pages are my own.

A note on unfamiliar terms

Abbreviations are spelled out the first time they're used in a chapter, but if you're dropping into a chapter out of order, Appendix A (Glossary) collects the recurring ones - SLO, SLA, SLI, MTTR, RPO/RTO, ADR, and the rest - in one place, each with a pointer back to the chapter that covers it properly.

Chapter 8 - Safe Schema Evolution at Scale

The migration that only failed in production

On Northfield, a fictional order-processing platform, a routine cleanup task renames a column: `orders.buyer_email` becomes `orders.customer_email`, to match naming used everywhere else in the domain. The migration is small, obviously correct, and reviewed without objection. In staging, against a ten-thousand-row table, it runs in forty milliseconds. In production, against a table with two hundred million rows, the `ALTER TABLE` takes an exclusive lock for twenty-five minutes. Every write to `orders` blocks behind it. The API starts returning timeouts, the on-call engineer gets paged, and the migration that "obviously" worked has to be killed mid-run, leaving the schema in an ambiguous state.

Nothing about the migration was wrong, syntactically. It was tested at the wrong scale, the same failure mode as Chapter 7, applied to schema changes instead of queries. This chapter covers the patterns that make schema changes safe regardless of table size: non-blocking DDL, a backward-compatible rename pattern, batched backfills, and - the part teams forget most often - tracking who else reads your schema before you assume a "safe" change is actually safe for everyone.

Why "fast in staging" doesn't transfer

The cost of a schema change is a function of data volume, not query complexity, and that cost varies by database engine and by the specific operation:

- Adding a nullable column with no default is metadata-only and near-instant on both MySQL (InnoDB, modern versions) and PostgreSQL, regardless of table size.
- Adding a column with a default value, or adding a `NOT NULL` constraint to an existing column, historically required rewriting every row to populate or validate the new value - a cost proportional to table size. Recent versions of both engines have narrowed this (PostgreSQL avoids a full rewrite for constant defaults since version 11; MySQL's behavior depends on the specific `ALGORITHM` used), but the safe assumption is: verify the specific operation against the specific engine version in use, on a copy of production-sized data, before trusting it in staging.

- Renaming a column, dropping a column, or changing a column's type can each range from instant metadata changes to full table rewrites, again depending on engine and version.

The rule that survives all of that engine-specific detail: never trust a migration's timing from a small staging database. Run it against a production-sized copy (a recent snapshot, restored somewhere disposable) before it ships, and know in advance whether the specific operation takes a blocking lock.

Concretely, adding a `NOT NULL` constraint to `orders.customer_email` on PostgreSQL as a single step forces the database to scan and validate every one of two hundred million rows while holding a lock that blocks writes for the duration:

```
// Locks orders for the full validation scan - the whole table, all at
once.
Schema::table('orders', function (Blueprint $table) {
    $table->string('customer_email')->nullable(false)->change();
});
```

Splitting it into "add the constraint without enforcing it yet" and "validate it separately" turns one long exclusive lock into a near-instant metadata change plus a scan that only takes a brief lock at the very end:

```
-- Step 1: instant - takes the constraint's existence for granted, doesn't
scan.
ALTER TABLE orders ADD CONSTRAINT customer_email_not_null
    CHECK (customer_email IS NOT NULL) NOT VALID;

-- Step 2: scans the table, but only takes a brief lock to flip the
-- constraint's state once the scan finds no violations - not for the
-- duration of the scan itself.
ALTER TABLE orders VALIDATE CONSTRAINT customer_email_not_null;
```

Run as two separate deploys, the first one is safe to ship immediately; the second can run whenever the backfill from the next section has finished, with no code depending on either step's timing.

Concurrent and non-blocking index creation

Adding an index to a large table is one of the most common migrations to get wrong, because a plain `CREATE INDEX` takes a lock that blocks writes for as long as the index build takes - which, on a two-hundred-million-row table, can be tens of minutes.

PostgreSQL's answer is `CREATE INDEX CONCURRENTLY`, which builds the index without holding a write lock, at the cost of a slower two-pass build and a real failure mode: if it's interrupted, it can leave behind an invalid index that has to be dropped and retried, not resumed.

```
CREATE INDEX CONCURRENTLY idx_orders_customer_created
    ON orders (customer_id, created_at);
```

MySQL's InnoDB engine supports online DDL for many index operations via `ALGORITHM=INPLACE, LOCK=NONE`, which allows concurrent reads and writes during the build - but not every DDL operation supports every algorithm, and the safe habit is to check the specific operation against the running engine version rather than assume.

```
ALTER TABLE orders
    ADD INDEX idx_orders_customer_created (customer_id, created_at),
    ALGORITHM=INPLACE, LOCK=NONE;
```

Laravel's migration files don't manage this distinction automatically - a `Schema::table()` migration using `$table->index()` issues a plain `CREATE INDEX`/`ADD INDEX` unless the raw SQL or a database-specific driver option is used to request the non-blocking variant. On a large table, that's a decision to make explicitly, not a default to trust:

```
public function up(): void
{
    // Schema::table()->index() takes a blocking lock for the build - fine
    // on a small table, a production incident on a two-hundred-million-
row one.
    DB::statement(
        'CREATE INDEX CONCURRENTLY idx_orders_customer_created ON orders
(customer_id, created_at)'
    );
}
```

`CREATE INDEX CONCURRENTLY` can't run inside the transaction Laravel wraps each migration in by default, so the migration class also needs `public $withinTransaction = false;` - forgetting that is the most common way this pattern fails silently in a migration file, even though it works fine when run by hand. And because an interrupted concurrent build leaves an unusable index behind rather than rolling back, always check for one before retrying:

```
-- Finds indexes left in an invalid state by an interrupted CONCURRENTLY
build.
SELECT indexrelid::regclass AS index_name
FROM pg_index
WHERE indisvalid = false;

-- Safe to drop and retry - an invalid index is never used by the planner.
DROP INDEX CONCURRENTLY idx_orders_customer_created;
```

The backward-compatible column rename pattern

Renaming a column directly - `ALTER TABLE orders RENAME COLUMN buyer_email TO customer_email` - is fast as a pure metadata operation on most engines. The real danger isn't the `ALTER` itself; it's that the moment it commits, every piece of code still referencing the old column name breaks atomically, everywhere, at once - including code you don't control (see the next section). The fix is to never do a rename as a single step. Expand it into phases that can each be shipped, verified, and rolled back independently:

Phase	Schema state	Application state	Rollback if something's wrong
1. Add	New column exists, nullable	Not yet used	Drop the new column - no impact
2. Dual-write	Both columns exist	App writes to both columns on every write path; reads still from the old column	Stop dual-writing - old column is still authoritative
3. Backfill	Both columns exist	Batch job populates the new column for existing rows	Backfill is idempotent - safe to re-run or pause
4. Migrate reads	Both columns exist	App reads from the new column; old column no longer read	Revert the read change - old column is still fully populated
5. Stop dual-write	Both columns exist	App writes only to the new column	Revert to dual-writing if any consumer still expects the old column
6. Drop	Old column removed	Fully migrated	Not reversible past this point - this phase only ships once nothing reads the old column

Each phase is a separate, independently deployable change. If phase 4 reveals a bug (the new column has a subtly different meaning than assumed), the rollback is "revert one deploy," not "recover from a completed rename." This is the same expand-and-contract shape used for the shared-package upgrades in Chapter 6 - the pattern generalizes to any change where "old" and "new" need to coexist for a transition window.

Phase 1 is the only phase that touches the database schema directly - it's a plain, additive migration:

```
public function up(): void
{
    Schema::table('orders', function (Blueprint $table) {
$table->string('customer_email')->nullable()->after('buyer_email');     //
BTW after() doesn't work in postgres
    });
}
```

Phase 2's dual-write lives in the model, not in a migration, and is the one line of application code every write path now depends on until phase 5 removes it:

```
protected static function booted(): void
{
    static::saving(function (Order $order) {
        if ($order->isDirty('buyer_email')) {
            $order->customer_email = $order->buyer_email;
        }
    });
}
```

Phase 6's drop is deliberately its own migration, shipped only after the checklist at the end of this chapter is fully checked off:

```
public function up(): void
{
    Schema::table('orders', function (Blueprint $table) {
        $table->dropColumn('buyer_email');
    });
}
```

Batching large backfills

Phase 3 above - populating the new column for two hundred million existing rows - is itself a hazard if done as one statement:

```
// Don't do this on a large table.
DB::table('orders')->update(['customer_email' => DB::raw('buyer_email')]);
```

A single `UPDATE` touching every row holds row locks (and, depending on engine and isolation level, can hold them for the duration of one very long transaction), competes with live traffic for the same rows, and - on replicated setups - can create significant replication lag, since a replica applies the same enormous write as one unit. The safe alternative is chunking: touch a bounded number of rows per statement, with a small pause between batches to let replicas catch up and to leave room for live traffic.

```
Order::query()
    ->whereNull('customer_email')
    ->orderBy('id')
    ->chunkById(1000, function ($orders) {
        foreach ($orders as $order) {
            $order->update(['customer_email' => $order->buyer_email]);
        }

        usleep(100_000); // 100ms - bound replication lag and lock
contention
    });
```

Running this as a queued, resumable Artisan command (tracking the last processed ID) rather than a single migration step means it can be paused, monitored, and restarted without redoing completed work - important for a backfill that might run for hours against a genuinely large table:

```
final class BackfillCustomerEmail extends Command
{
    protected $signature = 'orders:backfill-customer-email {--fresh} {--
dry-run}';

    public function handle(): int
    {
        $lastId = $this->option('fresh') ? 0 : (int)
Cache::get('backfill:customer_email:last_id', 0);
```

```
        $dryRun = $this->option('dry-run');
        $processed = 0;

        Order::query()
            ->whereNull('customer_email')
            ->where('id', '>', $lastId)
            ->orderBy('id')
            ->chunkById(1000, function (Collection $orders) use ($dryRun,
&$processed) {
                if (! $dryRun) {
                    foreach ($orders as $order) {
                        $order->update(['customer_email' =>
$order->buyer_email]);
                    }

                    // Persisted, not just held in memory - a restart
after a
                    // deploy or an `artisan:down` picks up where this
left off
                    // instead of re-scanning rows already backfilled.
                    Cache::forever('backfill:customer_email:last_id',
$orders->last()->id);
                }

                $processed += $orders->count();

                // Without this, a backfill that runs for hours looks
                // identical to a stuck one - there's nothing to
distinguish
                // "still working" from "silently hung" on the terminal.
                $this->components->info(
                    ($dryRun ? '[dry run] ' : '')."Backfilled {$processed}
rows so far (last ID: {$orders->last()->id})."
                );

                usleep(100_000);
            });

        $this->components->info($dryRun
            ? "Dry run complete: {$processed} rows would have been
backfilled."
```

```
            : 'Backfill complete: no rows remain with a null
customer_email.');

        return self::SUCCESS;
    }
}
```

Dispatched via `Schedule::command('orders:backfill-customer-email')->everyMinute()->withoutOverlapping()` (Chapter 10 covers scheduling and overlap guards in depth), this makes progress in small, bounded increments and survives being interrupted by a deploy at any point.

`--dry-run` is worth adding to every backfill command before it ever touches a large table: it walks the exact same query and chunking logic, reports how many rows and which ID range would be affected, and writes nothing - so a mistake in the `WHERE` clause shows up as a suspicious row count in dry-run output, not as two hundred million incorrectly updated rows. And it's always worth emitting a progress line per batch, not just a single message at the end - a backfill that runs for hours with no output is indistinguishable from one that's silently hung, and the per-batch line (row count so far, last ID reached) is what turns "is this still running?" into a question the terminal already answers.

Downstream consumers: your schema is a public API

The phased rename above protects the application. It does nothing for anything else that reads the same database directly and isn't part of the deploy you control: a nightly analytics export, another internal service with direct read access to the same tables (an anti-pattern, but a common inherited one), or a BI tool querying a read replica. From their perspective, dropping `buyer_email` in phase 6 is a breaking change shipped with zero notice, because nothing about the application's own migration process is aware they exist.

The fix starts with an inventory: before any schema change on a table of nontrivial age, identify who reads it besides the application - export jobs, other services, reporting queries, scheduled dashboards. For Northfield's rename, that inventory turned up an internal reporting export that ran nightly against `orders.buyer_email` directly, owned by a different team on a different release cycle.

Two patterns handle this without forcing every downstream consumer onto the application's timeline:

- **A compatibility view.** Keep a database view (or, during the dual-write window, the old column itself) that presents the old shape, so existing consumer queries keep working unmodified while the underlying table has already moved on. The view gets dropped only once every known consumer has confirmed it's no longer needed - not on the application's schedule, but on the slowest consumer's.
- **A deprecation window with an actual notification.** Whoever owns the schema change communicates the timeline to known downstream owners the same way a public API deprecates an endpoint: an announced window, not a silent drop. This only works if the inventory step actually happened - a consumer nobody knew about can't be notified.

Both patterns above are damage control for consumers that already read columns directly. The more durable fix, when the downstream consumer is an HTTP client rather than a direct database reader, is to never let that exposure exist in the first place: don't return `Model::toJson()` or a bare `$order->toArray()` from a controller, and don't let a resource's field names default to mirroring column names. A Laravel API Resource (or a `league/fractal` transformer, on a project that predates or doesn't use Resources) sits between the two and makes the wire format an explicit, independently versioned contract:

```
final class OrderResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            // The response field stays `buyer_email` - the contract every
            // existing API client already depends on - even though the
            // column behind it is `customer_email` after phase 6 ships.
            // The rename happens entirely on one side of this line.
            'buyer_email' => $this->customer_email,
            'status' => $this->status,
            'total_cents' => $this->total_cents,
        ];
    }
}
```

With that boundary in place, the entire six-phase rename in this chapter can happen without a single HTTP API consumer noticing - the resource is the only place that has to change, and it changes in the opposite direction: mapping the new column to the response field the API already promised, rather than the API being what dictates the column name.

For the rename, the old `buyer_email` column stayed in place, populated by the dual-write step, until the reporting team confirmed their nightly export had been updated to the new column name - on their schedule, not the application's. Only then did phase 6 ship. Had the export needed more time than the dual-write window allowed, a compatibility view would have covered the gap:

```
-- Ships in the same deploy as phase 6's drop, so the reporting export's
-- `SELECT buyer_email FROM orders` keeps working unmodified, reading
-- from the new column under an old name it still recognizes.
CREATE VIEW orders_legacy_shape AS
    SELECT orders.*, customer_email AS buyer_email
    FROM orders;
```

A schema-change checklist

```
- [ ] Tested the specific operation (not just "a migration") against a
production-sized copy of the table
- [ ] Confirmed whether the operation takes a blocking lock on the current
engine/version
- [ ] Used a non-blocking index build (CONCURRENTLY / ALGORITHM=INPLACE)
for large tables
- [ ] Renames/type changes are split into add -> dual-write -> backfill ->
migrate reads -> drop phases
- [ ] Backfills run in bounded batches with a pause, not as one statement
- [ ] Inventoried downstream consumers (exports, other services, BI tools)
that read this table directly
- [ ] HTTP API responses go through a Resource/transformer, not a bare
model - column renames don't reach API clients
- [ ] Downstream consumers notified or bridged with a compatibility view
before the old shape is dropped
- [ ] The "drop" phase is the last, separately reviewed step - never
bundled with the rest
```

Chapter recap

- A migration's cost scales with table size and the specific engine/operation, not with how simple the change looks in code - always test DDL against production-sized data.
- Use non-blocking index builds (`CREATE INDEX CONCURRENTLY` on PostgreSQL, `ALGORITHM=INPLACE, LOCK=NONE` on MySQL) on any table large enough that a lock would be noticed.
- Never rename or retype a column in one step - expand it into add, dual-write, backfill, migrate reads, stop dual-write, and drop, each independently deployable and reversible until the last phase.
- Backfill large tables in bounded, pausable batches (`chunkById` plus a small delay), not a single sweeping `UPDATE`, to protect replication and live traffic.
- Your schema is effectively a public API the moment anything outside your own deploy - an export, another service, a BI tool - reads it directly. Inventory those consumers before assuming a change is safe.
- For consumers that go through HTTP rather than the database directly, a Laravel API

Resource (or `league/fractal`) makes the response shape an explicit contract instead of a mirror of column names - the safest fix is the one that prevents the exposure from existing at all.

- Bridge downstream consumers with a compatibility view or an announced deprecation window; let the slowest consumer's timeline, not your migration's convenience, decide when the old shape can actually be dropped.

Test type	What it does	What it answers
Load test	Sustained traffic at (or near) expected peak, realistic mix, realistic ramp	"Do we meet our SLOs at the volume we expect?"
Stress test	Push volume past expected peak until something breaks	"Where's the ceiling, and what breaks first when we exceed it?"
Soak test	Moderate, sustained load over hours, not minutes	"Do we leak memory, connections, or disk over time under continuous load?"
Spike test	Sudden jump from baseline to peak with no ramp	"Does autoscaling/caching keep up with instantaneous demand, or is there a painful cold-start window?"

A pre-launch checklist for a known high-traffic event usually wants all four, in that order: load first (does the expected case work at all), stress second (how much headroom exists above the forecast, in case the forecast is wrong - and it usually is), soak third (does the system degrade over the multi-hour window the event actually runs for), and spike last (does the traffic curve's steepest section, not just its peak, hold up).

Finding the real bottleneck instead of guessing at it

A load test's most valuable output usually isn't the pass/fail verdict, it's the trail of evidence for *where* the system degraded first. That requires instrumenting every layer the request touches during the test, not just watching the application's own response time:

- **Database.** Connection pool saturation, slow query log entries that only appear under concurrency (lock waits, not just slow individual queries), replica lag if reads are offloaded.
- **Queue.** Job backlog depth over time - a queue that's "keeping up" at low load can start accumulating a backlog the moment throughput crosses the workers' effective processing rate, and a backlog that only grows never recovers on its own (see Chapter 10).

- **Cache.** Hit rate under the test's actual access pattern - a synthetic test that hits the same ten product IDs repeatedly will show a hit rate no real, long-tail catalog traffic will ever produce.
- **External dependencies.** Every third-party call the request path makes, with its own latency and error tracking. This is where the flash-sale scenario's real answer lives: the application layer holds up fine at ten times normal traffic, but the payment gateway's own published rate limit is lower than the target checkout throughput, so a fraction of legitimate checkouts start getting rejected by the gateway itself, not by anything in the platform's own control. No amount of application-side tuning fixes that; the fix is negotiating a higher limit with the vendor, queuing/smoothing checkout submissions client-side, or accepting a lower throughput ceiling and communicating it to the business ahead of time - all decisions that need to be made weeks before the event, not discovered during it.

The general principle: **load-test with the same observability you'd want during a real incident.** If a metric isn't visible during the test, the test can only tell you *that* something broke, not *why* - and "why" is the only actionable output.

From test results to a capacity plan

A load test tells you what happened at the load you tested. A capacity plan turns that into: how much load can we actually take, how do we know when we're approaching it, and what do we do next.

- **Headroom, not just pass/fail.** If SLOs hold at 2,000 requests per second, push further in the stress test until they don't, and record that ceiling. "We pass at target load" and "we pass at target load with 3x headroom before failure" imply very different risk postures for the actual event.
- **Leading indicators, not lagging ones.** Translate the bottleneck found during testing into a dashboard metric and an alert threshold that fires *before* users are affected - queue depth crossing a threshold, connection pool utilization crossing 80%, gateway error rate ticking up - so the response happens during the ramp, not after the SLO has already been breached.
- **A pre-agreed response plan.** Decide, in advance, what happens if a leading indicator fires during the real event: is there a feature flag to disable a non-critical path, a manual scaling trigger, a fallback for the constrained dependency? Writing this down

before the event turns a live incident into "run the plan" instead of "improvise under pressure."
- **Revisit before the next event, not after.** Capacity plans rot. The catalog grows, the query patterns shift, a new feature added a database call to the checkout path since the last test. Treat load testing as a recurring practice ahead of every high-stakes traffic event, not a one-time certification.

A leading-indicator check like the queue-depth example above might run as a scheduled job, translating the threshold into something that actually fires:

```
// Runs on a schedule, checking the bottleneck found during load testing.
$queueDepth = Queue::size('checkout');

if ($queueDepth > 5000 && ! Cache::has('checkout-queue-alert-cooldown')) {
    Notification::route('slack', config('alerts.oncall_webhook'))
        ->notify(new QueueDepthExceeded('checkout', $queueDepth));

    Cache::put('checkout-queue-alert-cooldown', true,
now()->addMinutes(10));
}
```

The threshold and cooldown are tuned per dependency, but the shape - check, compare, notify once - stays the same.

Chapter recap

- Write SLOs (latency percentiles, error rate ceiling, throughput target grounded in a real forecast) before running a load test, or "passing" has no meaning.
- Model realistic traffic: a weighted mix of user journeys with think time and a ramp that matches the expected arrival curve, not a single endpoint hammered at constant concurrency.
- Load, stress, soak, and spike tests answer different questions - use the one that matches the risk you're actually worried about, and run all four ahead of a genuinely high-stakes event.
- Instrument every layer the request touches (database, queue, cache, external dependencies) during the test, because the real bottleneck - like a third-party payment

gateway's own rate limit - is often outside the application layer entirely and invisible if you only watch your own response times.

- Turn results into leading-indicator alerts and a pre-agreed response plan, and re-run the test before the next high-traffic event, not just the first one.

Part 3 - Correctness Under Load

Performance and availability don't matter if the system produces wrong answers under concurrency - idempotency, state machines, ledger patterns, and event sourcing for systems where a retried request or a race condition can't be shrugged off, because the mistake is money.

Chapter 14 - Money, State Machines, and Idempotency

The webhook that fired twice

Vaultly, a fictional wallet platform, integrates a payment gateway for card top-ups. The gateway does what every reputable payment provider does: it sends a webhook when a charge settles, and if it doesn't get a fast `200` back, it retries - at 30 seconds, then a minute, then five minutes, for up to 24 hours, because from the gateway's point of view a timeout is indistinguishable from a lost request, and losing a "your charge succeeded" notification is a worse failure than sending it twice.

On a routine day, a webhook handler is mid-way through crediting a user's balance when a deploy restarts the app server. The database write never happens, so the handler never returns `200`, so the gateway retries thirty seconds later - and this time it succeeds cleanly, credits the balance, returns `200`. Except the *first* attempt wasn't fully rolled back either: an earlier version of the code sent a "funds added" notification email as a side effect before writing to the database, and that side effect fired on the failed attempt too. Now support is fielding a confused user asking why they got two emails but only one balance update - a relatively benign version of a bug that, in a different code path, could just as easily have credited the account twice instead of once.

This is not a rare edge case; it's the default behavior of any reliable message delivery system (webhooks, queues, SQS, most message brokers), which the industry summarizes as **at-least-once delivery**: a message might be delivered more than once, but it won't silently vanish. The corresponding engineering obligation is **idempotency**: making sure that receiving the same logical event N times has the same effect as receiving it once. Money-shaped problems - anything where a duplicated write means a duplicated debit, credit, or shipment - can't tolerate skipping this step, and this chapter is about the three tools that make it tractable: transaction boundaries, idempotency keys, and explicit state machines.

Side effects don't belong inside "in progress"

The email-notification bug above has a specific, common shape: a side effect (send an email, call another API, publish an event) ran *before* the database transaction that represents "this actually happened" had committed. If that transaction later rolls back, or the request fails after the side effect but before the commit, the side effect already happened and can't be un-sent.

The fix is a rule, not a clever library: **only trigger irreversible side effects after the transaction that records them has committed**, and treat every side effect as something that might run more than once regardless.

```
DB::transaction(function () use ($charge) {
    $wallet = Wallet::lockForUpdate()->findOrFail($charge->wallet_id);
    $wallet->credit($charge->amount);
    $charge->markAsProcessed();
});

// Only after the commit succeeds - and even then, guard against
// this handler itself being re-invoked for the same charge.
event(new WalletCredited($charge));
```

`lockForUpdate()` matters here for a second, related reason: two webhook deliveries (or a webhook and an unrelated concurrent request touching the same wallet) racing to read-then-write a balance is a classic lost-update bug, independent of idempotency. Locking the row for the duration of the transaction serializes those writes so the second one sees the first one's result rather than overwriting it.

This gets the *ordering* right, but ordering alone doesn't stop the handler from running the whole block twice for the same charge. That's what idempotency keys are for.

Idempotency keys: making "already done" a fact the database can check

An idempotency key is a value that uniquely identifies *one logical operation*, supplied by whoever might retry it, and checked against a persistent record before the operation's effects

are applied a second time. For an inbound webhook, the gateway's own event ID is usually the key; for an outbound request your own system initiates (like a "process this payout" job that might be retried by the queue), you generate the key yourself and pass it through every retry of that same logical attempt.

The mechanism that makes this reliable isn't application logic checking "have I seen this before?" in a separate `if` statement - that has its own race condition when two deliveries arrive close together. It's a **unique constraint the database enforces**, so the "have we processed this" check and the "mark it processed" write happen atomically:

```
Schema::create('processed_webhook_events', function (Blueprint $table) {
    $table->id();
    $table->string('provider')->index();
    $table->string('event_id');
    $table->timestamp('processed_at');
    $table->unique(['provider', 'event_id']);
});
```

This is a sample from "Laravel in Production: Architecture, Performance, and Operations at Scale".